操作系统原理实验报告

实验名称	授课老师	学生姓名	学生学号
内核线程	张青	王健阳	21307261

实验要求

1. 了解C语言的可变参数机制
2. 接触并熟悉PCB的数据结构
3. 认识到线程的简单代码实现
4. 学习并实现一些简单调度算法

实验过程

Assignment 1 printf的实现

> 学习可变参数机制，然后实现printf，你可以在材料中的printf上进行改进，或者从头开始实现自己的printf函数。结果截图并说说你是怎么做的。

已实现的功能如下：

- `%%`。输出一个`%`。
- `%c`。输出`ap`指向的字符。
- `%s`。输出`ap`指向的字符串的地址对应的字符串。
- `%d`。输出`ap`指向的数字对应的十进制表示。
- `%x`。输出`ap`指向的数字对应的16进制表示。
- 其他。不做任何处理。

对于`%d`和`%x`，我们需要将数字转换为对应的字符串。一个数字向任意进制表示的字符串的转换函数如下所示，声明放置在`include/stdlib.h`中，实现放置在`src/utils/stdlib.cpp`中。

改进：允许形如%nd，%nc等形式的输入。n是正整数，含义是如果n小于等于输出字符或者数字长度则原样输出，否则将保持输出长度为n的字符串，不足的部分在左边用' '补足

```
int printf(const char *const fmt, ...)
{
```

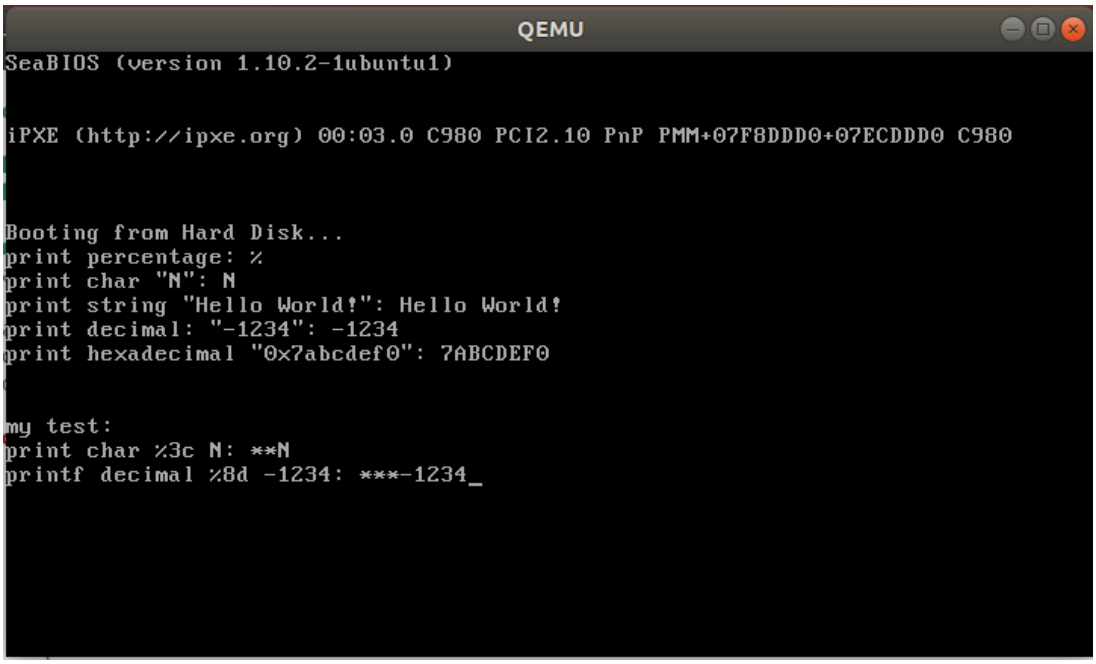
```
    const int BUF_LEN = 32;

    char buffer[BUF_LEN + 1];
    char number[33];

    int idx, counter;
    va_list ap;

    va_start(ap, fmt);
    idx = 0;
    counter = 0;

    for (int i = 0; fmt[i]; ++i)
    {
        //定义需要的变量
        int space_num = 0, num_len = 0, flag = 0;
        if (fmt[i] != '%')
        {
            counter += printf_add_to_buffer(buffer, fmt[i], idx, BUF_LEN);
        }
        else
        {
            i++;
            //读取所需的数字
            while (fmt[i] > '0' && fmt[i] <= '9'){
                space_num *= 10;
                space_num += fmt[i] - '0';
                i ++;
            }
            space_num -= 1;
            if (fmt[i] == '\0')
            {
                break;
            }

            switch (fmt[i])
            {
            case '%':
                counter += printf_add_to_buffer(buffer, fmt[i], idx, BUF_LEN);
                break;

            case 'c':
                //补足空格
                while (space_num > 0){
                    counter += printf_add_to_buffer(buffer, ' ', idx, BUF_LEN);
                    space_num --;
                }
                counter += printf_add_to_buffer(buffer, va_arg(ap, char), idx,
BUF_LEN);
                break;

            case 's':
                buffer[idx] = '\0';
                idx = 0;
```

```
                counter += stdio.print(buffer);
                counter += stdio.print(va_arg(ap, const char *));
                break;

            case 'd':
            case 'x':
                int temp = va_arg(ap, int);
                //设置标志位（是否输出负号）
                if (temp < 0 && fmt[i] == 'd')
                {
                    flag = 1;
                    temp = -temp;
                }

                itos(number, temp, (fmt[i] == 'd' ? 10 : 16));
                while(number[num_len]) {
                    num_len ++;
                }
                space_num -= num_len;
                //如果剩余位置大于0，则说明需要输出空格补足
                for (;space_num > 0; space_num --){counter +=
printf_add_to_buffer(buffer, ' ', idx, BUF_LEN);}
                if (flag){counter += printf_add_to_buffer(buffer, '-', idx,
BUF_LEN);}
                for (int j = 0; number[j]; ++j)
                {
                    counter += printf_add_to_buffer(buffer, number[j], idx,
BUF_LEN);
                }
                break;
            }
        }
    }

    buffer[idx] = '\0';
    counter += stdio.print(buffer);

    return counter;
}
```

```
extern "C" void setup_kernel()
{
    // 中断处理部件
    interruptManager.initialize();
    // 屏幕IO处理部件
    stdio.initialize();
    interruptManager.enableTimeInterrupt();
    interruptManager.setTimeInterrupt((void *)asm_time_interrupt_handler);
    //asm_enable_interrupt();
    printf("print percentage: %%\n"
           "print char \"N\": %c\n"
```

```
            "print string \"Hello World!\": %s\n"
            "print decimal: \"-1234\": %d\n"
            "print hexadecimal \"0x7abcdef0\": %x\n",
            'N', "Hello World!", -1234, 0x7abcdef0);
    printf("\n\nmy test:\n");
    printf("print char %%3c N: %3c\n", 'N');
    printf("printf decimal %%8d -1234: %8d", -1234);
    //uint a = 1 / 0;
    asm_halt();
}
```

改进后`printf`的输出结果如下：



（这里为了更好鉴别，空格暂时用*替代）

Assignment 2 线程的实现

> 自行设计PCB，可以添加更多的属性，如优先级等，然后根据你的PCB来实现线程，演示执行结果。

实验中的PCB已经囊括了PCB最核心的属性，故不再做过多修改。鉴于assignment 4设计的非抢占式算法，以下的`ticks`属性已无必要，故删去。同时真正赋予`priority`属性意义。（轮转算法只是设立了优先级，全程并没有用到）演示结果在assignment 4中展现。

```
struct PCB
{
    int *stack;                     // 栈指针，用于调度时保存esp
    char name[MAX_PROGRAM_NAME + 1]; // 线程名
    enum ProgramStatus status;      // 线程的状态
    int priority;                   // 线程优先级
    int pid;                        // 线程pid
    //int ticks;                      // 线程时间片总时间
    int ticksPassedBy;              // 线程已执行时间
    ListItem tagInGeneralList;      // 线程队列标识
```

```
    ListItem tagInAllList;          // 线程队列标识
};
```

Assignment 3 线程调度切换的秘密

操作系统的线程能够并发执行的秘密在于我们需要中断线程的执行，保存当前线程的状态，然后调度下一个线程上处理机，最后使被调度上处理机的线程从之前被中断点处恢复执行。现在，同学们可以亲手揭开这个秘密。

编写若干个线程函数，使用gdb跟踪`c_time_interrupt_handler`、`asm_switch_thread`等函数，观察线程切换前后栈、寄存器、PC等变化，结合gdb、材料中“线程的调度”的内容来跟踪并说明下面两个过程。

- 一个新创建的线程是如何被调度然后开始执行的。
- 一个正在执行的线程是如何被中断然后被换下处理器的，以及换上处理机后又是如何从被中断点开始执行的。

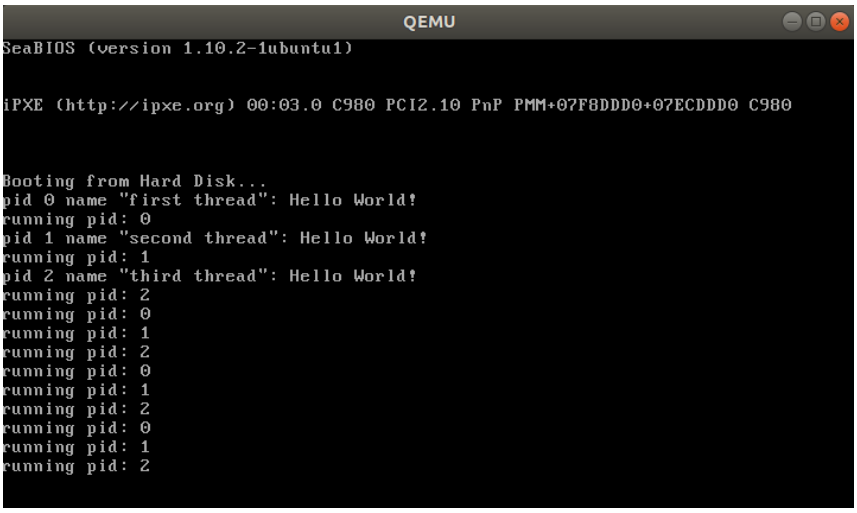
通过上面这个练习，同学们应该能够进一步理解操作系统是如何实现线程的并发执行的。

> 说明

这里为了实现真正意义上的时间片轮转，将三进程都设置了无限循环，最终效果如下：

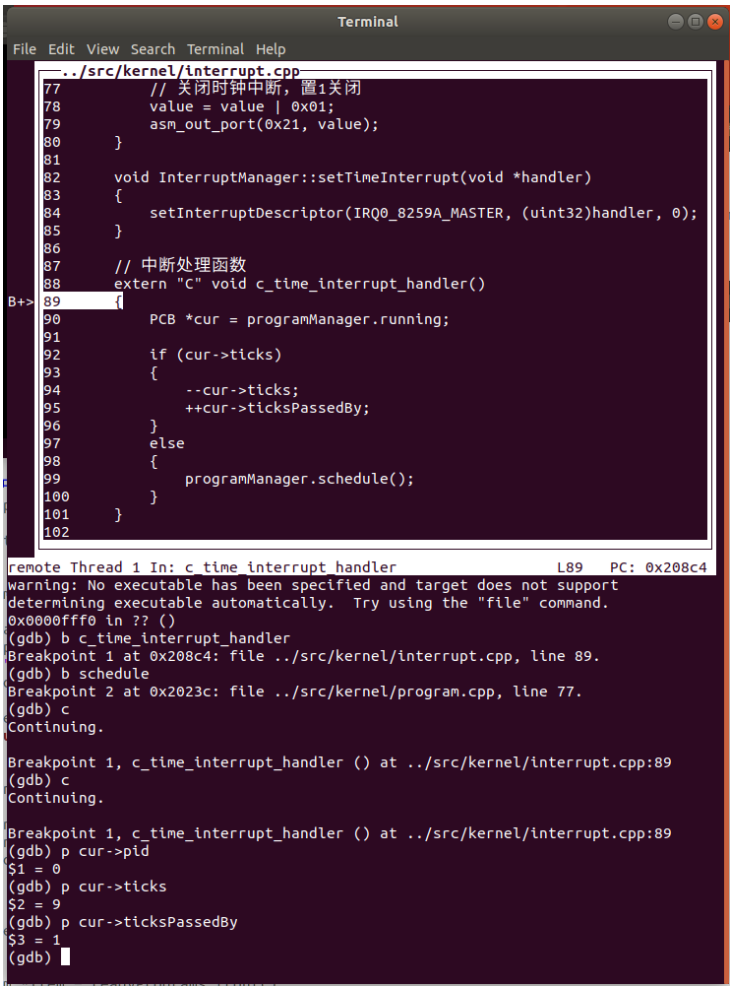
```
void third_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    while(1) {

    }
}
void second_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    while(1){

    }
}
void first_thread(void *arg)
{
    // 第1个线程不可以返回
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    if (!programManager.running->pid)
    {
        programManager.executeThread(second_thread, nullptr, "second thread", 1);
        programManager.executeThread(third_thread, nullptr, "third thread", 1);
    }
    asm_halt();
}
```

```
void ProgramManager::schedule()
{
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();
    printf("running pid: %d\n", running->pid);
    if (readyPrograms.size() == 0)
    {
        interruptManager.setInterruptStatus(status);
        return;
```



> c_time_interrupt_handler

每次到达时，打印出此时的线程pid，ticks以及ticksPassedBy。不难看出，对于单一进程，随着时间片的消耗，其ticks递减至0，然后便触发进程调度。



...

```
Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) p cur->pid
$4 = 0
(gdb) p cur->ticks
$5 = 8
(gdb) p cur->ticksPassedBy
$6 = 2
(gdb)
```

```
Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) p cur->pid
$10 = 0
(gdb) p cur->ticks
$11 = 0
(gdb) p cur->ticksPassedBy
$12 = 10
(gdb)
```

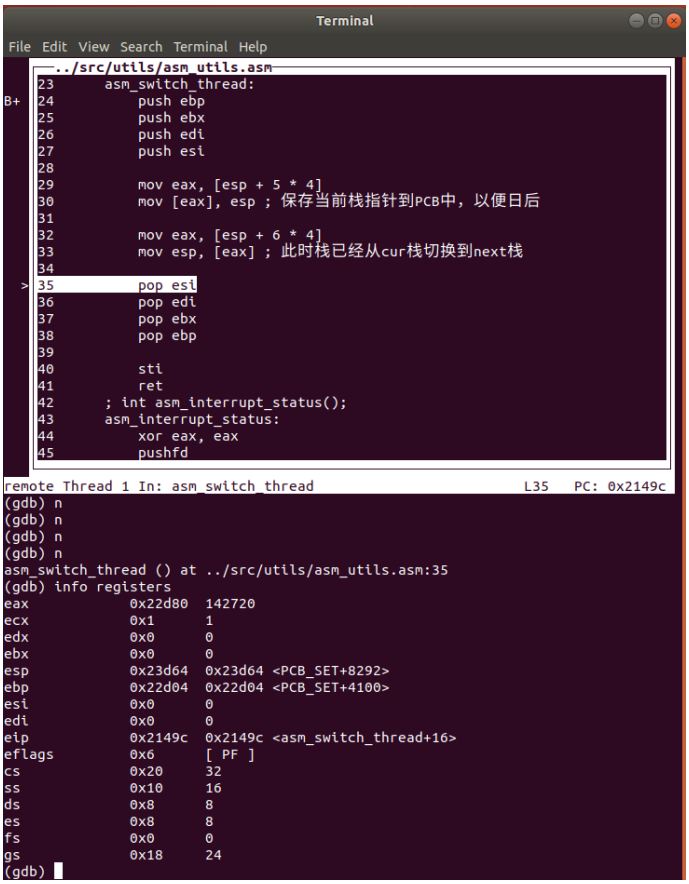
此时cur->ticks = 0，分配的时间片用完了，故准备进行进程调度

> asm_switch_thread

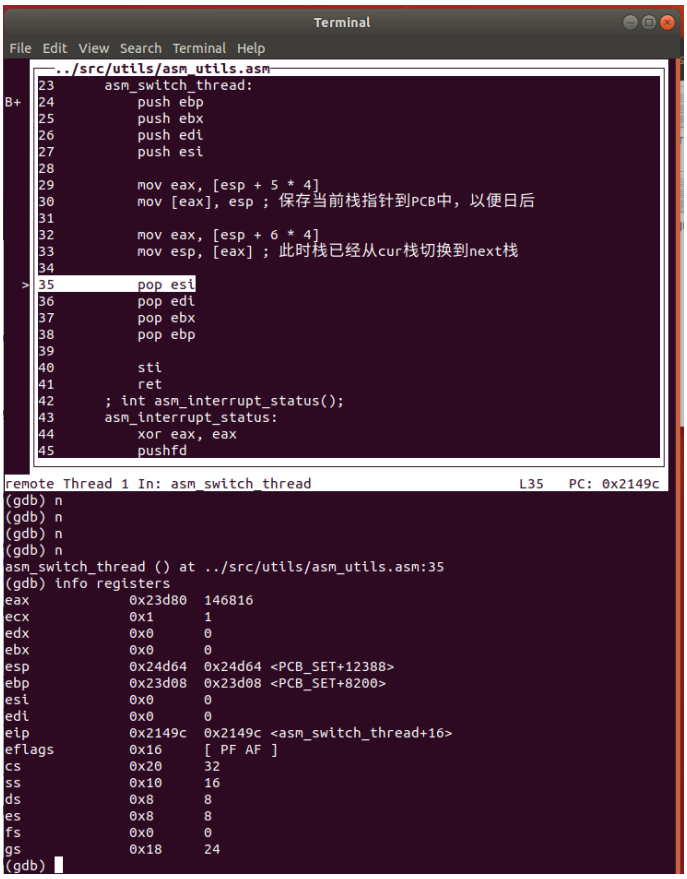
调度pid0

```
Terminal
File  Edit  View  Search  Terminal  Help
   ../src/utils/asm_utils.asm
   23      asm_switch_thread:
B+ 24          push ebp
   25          push ebx
   26          push edi
   27          push esi
   28
   29          mov eax, [esp + 5 * 4]
   30          mov [eax], esp ; 保存当前栈指针到PCB中，以便日后
   31
   32          mov eax, [esp + 6 * 4]
   33          mov esp, [eax] ; 此时栈已经从cur栈切换到next栈
   34
 > 35          pop esi
   36          pop edi
   37          pop ebx
   38          pop ebp
   39
   40          sti
   41          ret
   42      ; int asm_interrupt_status();
   43      asm_interrupt_status:
   44          xor eax, eax
   45          pushfd
remote Thread 1 In: asm_switch_thread                    L35   PC: 0x2149c
fs             0x0      0
gs             0x18     24
(gdb) n
(gdb) n
asm_switch_thread () at ../src/utils/asm_utils.asm:35
(gdb) info registers
eax            0x21d80  138624
ecx            0x21d80  138624
edx            0x0      0
ebx            0x39000  233472
esp            0x22d64  0x22d64 <PCB_SET+4196>
ebp            0x7bfc   0x7bfc
esi            0x0      0
edi            0x0      0
eip            0x2149c  0x2149c <asm_switch_thread+16>
eflags         0x12     [ AF ]
cs             0x20     32
ss             0x10     16
ds             0x8      8
es             0x8      8
fs             0x0      0
gs             0x18     24
(gdb)
```

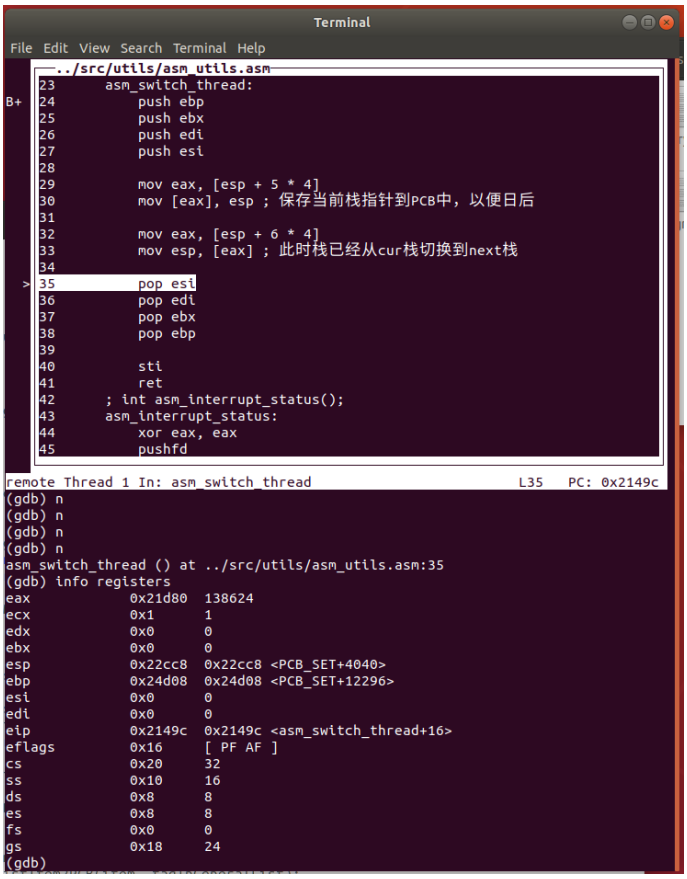
pid0下处理机，调度pid1



pid1下处理机，调度pid2



pid2下处理机，调度pid0



从中可以看出，一个新创建的线程会首先在`executeThread`中被分配相应的PCB，并对其内容进行初始化，然后将其加入就绪队列中等待调度（然后第一个程序是手动调度的），而从后面的进程开始，就是依次占用CPU资源直至时间片耗尽，在`c_time_interrupt_handler`函数中再次调用`schedule`实现线程调度。而对`asm_switch_thread`设置断点，则让我们清晰看到了PCB的切换（esp值的改变，线程栈的切换）。由于每次重新调度分配的PCB都是随机的，所以我们在pid2 -> pid0时看到的PCB跟一开始pid0的不太一样。

Assignment 4 调度算法的实现

在材料中，我们已经学习了如何使用时间片轮转算法来实现线程调度。但线程调度算法不止一种，例如

- 先来先服务。
- 最短作业（进程）优先。
- 响应比最高者优先算法。
- 优先级调度算法。
- 多级反馈队列调度算法。

此外，我们的调度算法还可以是抢占式的。

现在，同学们需要将线程调度算法修改为上面提到的算法或者是同学们自己设计的算法。然后，同学们需要自行编写测试样例来呈现你的算法实现的正确性和基本逻辑。最后，将结果截图并说说你是怎么做的。

以下均为非抢占式调度，故各个进程执行时间必须有限，故以下所有进程都不再包含死循环（包括pid0进程），对于进程的调度仅限于函数`program_exit`

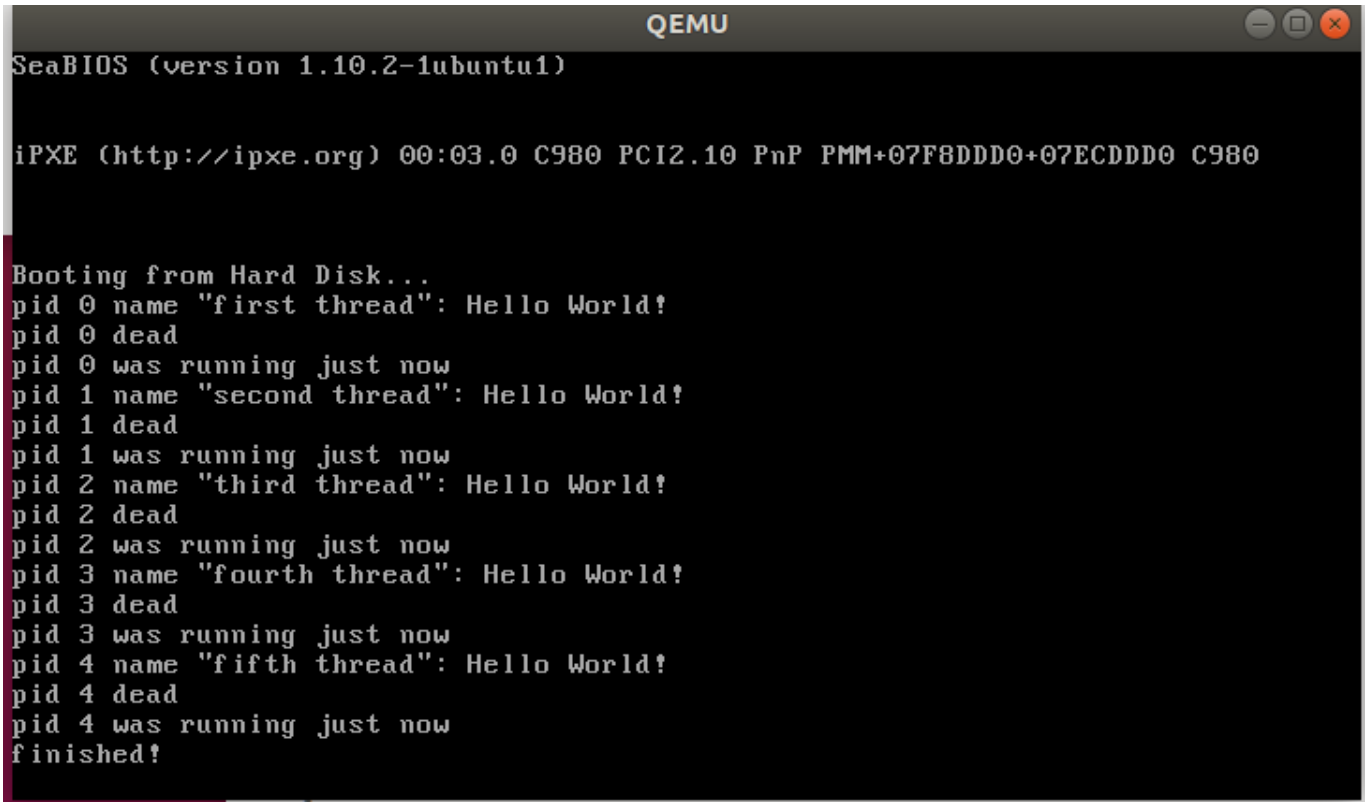
```
extern "C" void c_time_interrupt_handler()
{
    PCB *cur = programManager.running;
    ++ cur->ticksPassedBy; //记录进程执行时间
    //cur->priority += 1;
}
```

> 先来先服务算法

代码实现

```
void ProgramManager::schedule()
{
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();
    printf("pid %d was running just now\n", running->pid);
    if (readyPrograms.size() == 0)
    {
        interruptManager.setInterruptStatus(status);
        return;
    }

    if (running->status == ProgramStatus::DEAD)
    {
        releasePCB(running);
    }

    ListItem *item = readyPrograms.front();
    PCB *next = ListItem2PCB(item, tagInGeneralList);
    PCB *cur = running;
    next->status = ProgramStatus::RUNNING;
    running = next;
    readyPrograms.pop_front();

    asm_switch_thread(cur, next);

    interruptManager.setInterruptStatus(status);
}

void program_exit()
{
    PCB *thread = programManager.running;
    thread->status = ProgramStatus::DEAD;
    printf("pid %d dead\n", thread->pid);
    programManager.schedule();
    printf("finished!\n");
}
```

结果



按照预想的依次调用了4个线程

> 优先级算法

代码实现

```
void ProgramManager::schedule()
{
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();
    printf("pid %d was running just now\n", running->pid);
    if (readyPrograms.size() == 0)
    {
        interruptManager.setInterruptStatus(status);
        return;
    }

    if (running->status == ProgramStatus::DEAD)
    {
        releasePCB(running);
    }
    // 获取到就绪队列中优先级最高的进程
    ListItem *best = nullptr;
    int max_priority = -1;
    ListItem *item = readyPrograms.front();
    while(item){
        PCB *temp = ListItem2PCB(item, tagInGeneralList);
        if (max_priority < temp->priority){
            max_priority = temp->priority;
            best = item;
        }
```

```
        item = item->next;
    }
    // 删除相应进程
    readyPrograms.erase(best);
    PCB *next = ListItem2PCB(best, tagInGeneralList);
    PCB *cur = running;
    next->status = ProgramStatus::RUNNING;
    running = next;

    asm_switch_thread(cur, next);

    interruptManager.setInterruptStatus(status);
}

void program_exit()
{
    PCB *thread = programManager.running;
    thread->status = ProgramStatus::DEAD;
    printf("pid %d dead\n", thread->pid);
    programManager.schedule();
    printf("finished!\n");
}
```

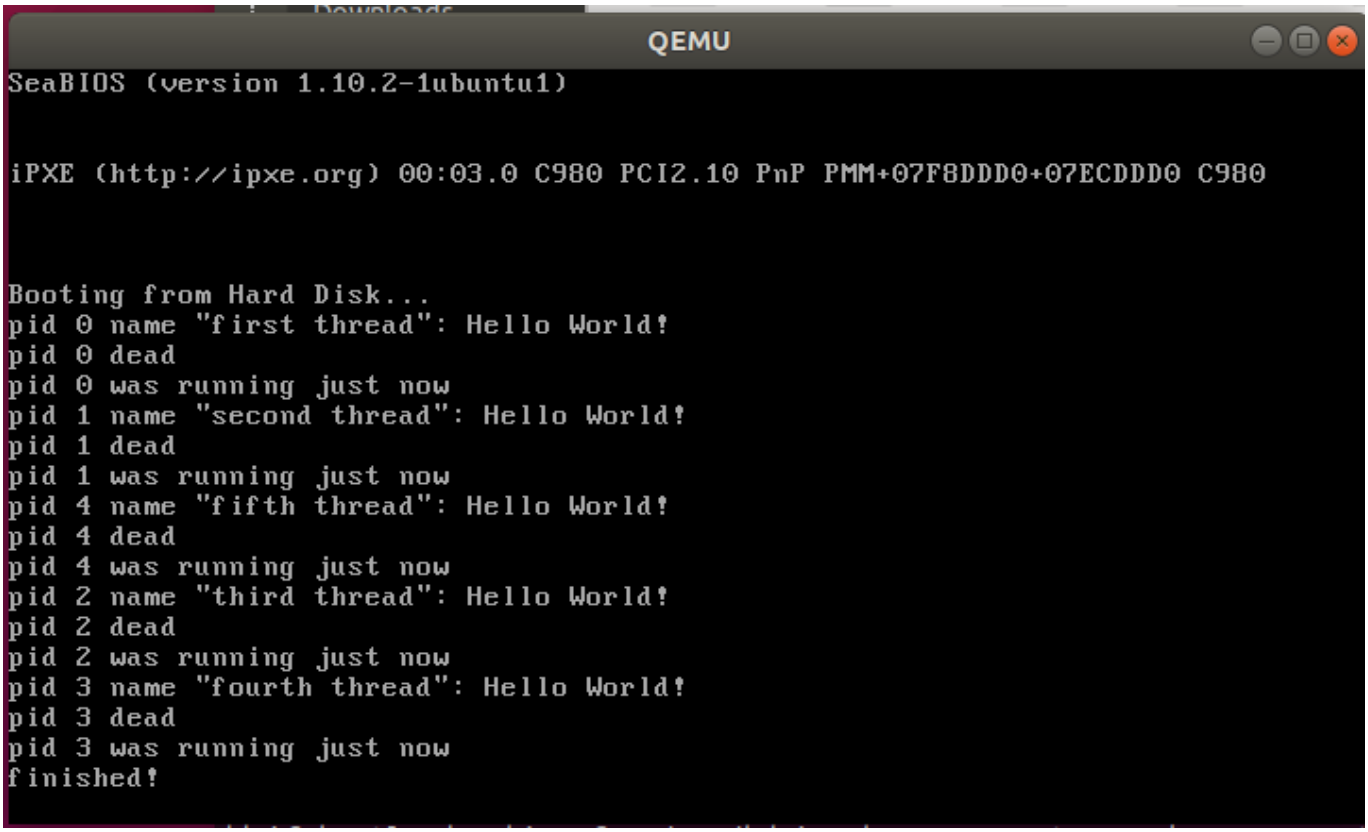
进程优先级设置如下：

```
#include "asm_utils.h"
#include "interrupt.h"
#include "stdio.h"
#include "program.h"
#include "thread.h"

// 屏幕IO处理器
STDIO stdio;
// 中断管理器
InterruptManager interruptManager;
// 程序管理器
ProgramManager programManager;

void third_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    while(0) {

    }
}
void second_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    while(0){

    }
}
```

```
void fourth_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    while(0){

    }
}
void fifth_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    while(0){

    }
}

void first_thread(void *arg)
{
    // 第1个线程不可以返回
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid,
programManager.running->name);
    if (!programManager.running->pid)
    {
        programManager.executeThread(second_thread, nullptr, "second thread", 4);
        programManager.executeThread(third_thread, nullptr, "third thread", 2);
        programManager.executeThread(third_thread, nullptr, "fourth thread", 1);
        programManager.executeThread(third_thread, nullptr, "fifth thread", 3);
    }
    //asm_halt();
}

extern "C" void setup_kernel()
{

    // 中断管理器
    interruptManager.initialize();
    interruptManager.enableTimeInterrupt();
    interruptManager.setTimeInterrupt((void *)asm_time_interrupt_handler);

    // 输出管理器
    stdio.initialize();

    // 进程/线程管理器
    programManager.initialize();

    // 创建第一个线程
    int pid = programManager.executeThread(first_thread, nullptr, "first thread",
100);
    if (pid == -1)
    {
        printf("can not execute thread\n");
        asm_halt();
    }

    ListItem *item = programManager.readyPrograms.front();
```

```
        PCB *firstThread = ListItem2PCB(item, tagInGeneralList);
        firstThread->status = RUNNING;
        programManager.readyPrograms.pop_front();
        programManager.running = firstThread;
        asm_switch_thread(0, firstThread);
        asm_halt();
    }
```

结果



最终进程的执行依旧符合预期。最开始的父进程优先级设置在了100，其余的子进程按照优先级4321递减依次执行。最后成功执行完毕所有线程。

总结

此次实验让我对进程调度有了进一步的了解，阅读C代码的能力进一步提升。一开始阅读如此大量的代码还是不太习惯，debug方面也是无从下手。借助这次实验的契机，我再次得到了一次锻炼使用gdb的机会，并在assignment 4 设计调度算法的时候成功用其找出了几个隐蔽的bug。这次实验也是收获颇丰。